CSS

{ ; }

הרצאה 3 • להוציא את העיצוב מתוך התגיות

צד לקוח • יעל סלע זעירא

היום בהרצאה

01 **מה זה CSS**
התחביר ואיפה כותבים אותו

02 **סלקטורים**
תג, `class`, `id` ושילובים

03 **Cascade ו-Specificity**
מי מנצח כשיש כמה חוקים

04 **סלקטורים מתקדמים**
`:nth-child`, `:hover`, > ועוד

05 **צבעים, גדלים וקובץ חיצוני**
`px` מול `em`, משתנים, `DevTools`

06 **box-sizing וסמנטיקה**
ARIA- ו-SEO

01

מה זה CSS

Cascading Style Sheets

Cascading Style Sheets

שפת עיצוב

מפרידה בין **התוכן** (HTML) לבין **המראה** (CSS)

Look & Feel

צבעים, גופנים, מרווחים, פריסה

קלה ללימוד

התחביר קטן - שלושה חלקים בלבד

כלי חזק מאוד

אותו HTML יכול להיראות אחרת לגמרי

בלי CSS היינו כותבים style="..." על כל תגית בנפרד - כמו שעשינו בהרצאה 1

יתרונות

חוסך זמן
חוק אחד משפיע על עשרות אלמנטים

הדף נטען מהר יותר
פחות קוד בכל דף, והדפדפן שומר את הקובץ ב-`cache`

קל לתחזוקה
שינוי צבע במקום אחד משנה את כל האתר

עיצוב עשיר יותר
אנימציות, מעברים ופריסות שאין ב-`HTML` לבד

סטנדרט
כל הדפדפנים מיישמים אותו

שלוש אפשרויות

איפה כותבים CSS

inline

תכונת `style` על התגית עצמה

מה שעשינו בהרצאה - 1לא מומלץ

internal

תגית `<style>` בתוך ה-`<head>`

טוב לדף בודד -נעבוד כך היום

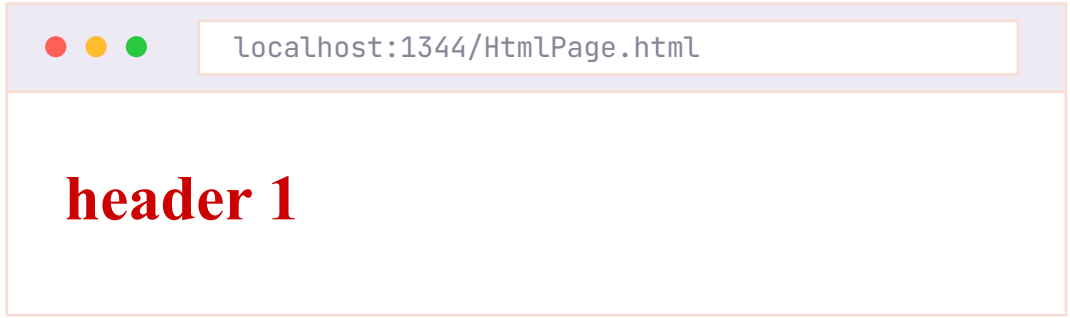
external

קובץ `.css` נפרד שמקושר לדף

הדרך הנכונה לאתר אמיתי

הפרדת העיצוב מה HTML וריכוז כללי עיצוב משותפים מצמצמים כפילויות ומקלים על התחזוקה

תגית <style> בתוך ה-<head>




```
<html>
<head>
    <meta charset="utf-8" />
    <title></title>
    <style>
        h1 {
            color: red;
        }
    </style>
</head>
<body>
    <h1>header 1</h1>
</body>
</html>
```

- התגית <style> יושבת תמיד בתוך ה-<head>, לא ב-<body>.
- כל מה שבתוכה הוא CSS - לא HTML.
- בהמשך ההרצאה נוציא את התוכן הזה לקובץ חיצוני.

התחביר

```
selector {
    property: value;
}
```

selector
על מי חל העיצוב

property
איזו תכונה משנים

value
לאיזה ערך

דוגמה: `h1 { color: red; }` - כל תגית `h1` בדף תקבל טקסט אדום.

02

סלקטורים

תג • class • id

שלושת סוגי הסלקטורים

סלקטור tag `h1`

חל על **כל** התגיות מהסוג הזה

סלקטור class `.redclass`

חל על כל מי שקיבל `class="red"`

סלקטור id `#myH1id`

חל על אלמנט **יחיד** בדף

```
h1 {
    color: red;
}

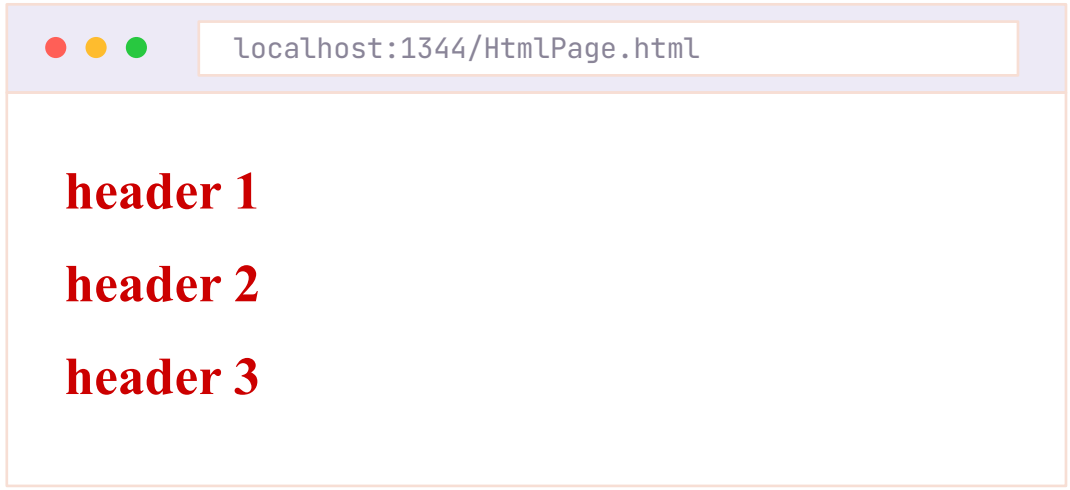
.red {
    color: red;
}

#myH1 {
    color: red;
}
```

שימו לב לתו הפותח: אין תו לתג, נקודה ל-`class`, סולמית ל-`id`.

סלקטור תג

העיצוב חל אוטומטית על **כל** התגיות מהסוג הזה -בלי לגעת ב-HTML בכלל:




```css
h1 {
    color: red;
}
```

בהרצאה 1 היינו צריכים לכתוב `style="color:red"` על כל `h1` בנפרד .עכשיו - חוק אחד.

הגדרה פעמיים לאותו תג




```
h1 {
    color: red;
}

h1 {
    color: blue;
    background-color: yellow;
}
```

- הצבע הוא **כחול** - התכונה `color` נדרסה על ידי ההגדרה השנייה.
- הרקע **צהוב** - `background-color` הוגדר רק פעם אחת ,ולכן נשאר.
- המסקנה : כשדורסים תכונה ,הערך האחרון הוא זה שקובע.

כמה תגים בבת אחת

אפשר להגדיר בפעם אחת מספר תגים שונים ,מופרדים בפסיק:

```
h1, p {
    color: red;
}
```



שכחתם פסיק? `h1 p` פירושו משהו אחר לגמרי - נראה את זה בהמשך.

ומה אם לא כל התגיות?

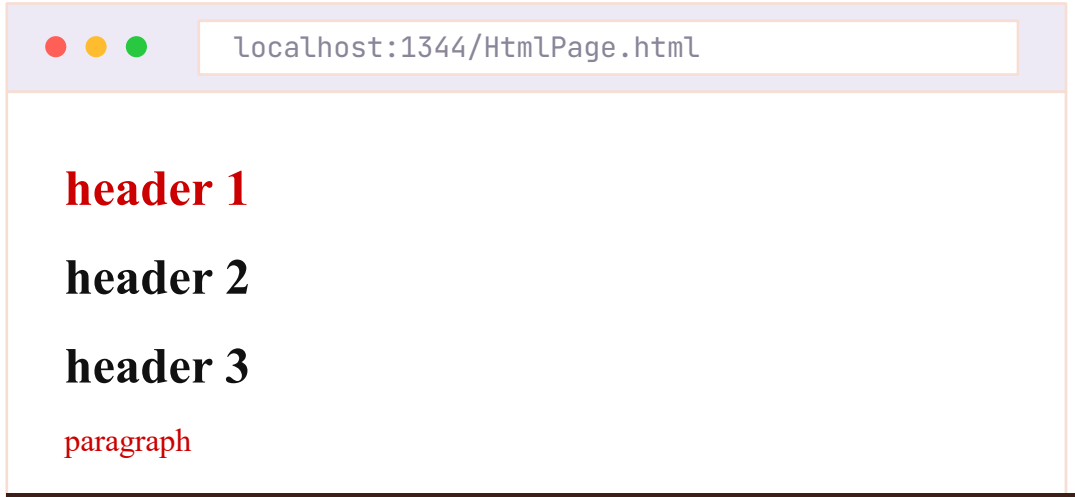
- מה קורה אם רוצים לעצב **רק** את הפסקה ואת הכותרת הראשונה?
- כלומר -להתייחס לקבוצה של אלמנטים, בלי קשר לסוג שלהם.
- לשם כך יש את התכונה `class` :נותנים לכל מי שרוצים את אותו שם מחלקה.

```
<h1 class="red">header 1</h1>
<h1>header 2</h1>
<h1>header 3</h1>

<p class="red">paragraph</p>
```

שם המחלקה נתון לבחירתכם -כדאי שיהיה בעל משמעות.

שימוש במחלקה




```css
.red {
    color: red;
}
```

שימו לב לנקודה . לפני שם המחלקה - זה מה שאומר לדפדפן שמדובר במחלקה ולא בשם של תגית.

אותה מחלקה יכולה לשבת על תגיות מסוגים שונים -כאן על h1 ועל p.

סלקטור id




```
#myH1 {
    color: red;
}


<h1>header 1</h1>
<h1 id="myH1">header 2</h1>
<h1>header 3</h1>
```

- מוסיפים לאלמנט תכונה `id` ונותנים לה שם באנגלית.
- בגיליון העיצוב פונים אליו עם סולמית `#`.
- `id` הוא **ייחודי** -אסור ששני אלמנטים יקבלו את אותו `id`.

שילוב סוגי סלקטורים

אפשר לשלב בשורה אחת סלקטורים מסוגים שונים, כל עוד התכונות שלהם זהות:

```
#myH1, .red {
      color: red;
}
```

תרגיל כיתה -CSS

- יישמו את העיצוב שמימין **בלי** להשתמש ב-`style` על התגיות.
- אלמנט יכול לקבל עיצוב מכמה מקורות: מהתג, מ-`class` ומ-`id`.
- חשבו איזה סלקטור מתאים לכל חלק.



03

למה "מדורגים?"

Cascade • Specificity • הורשה

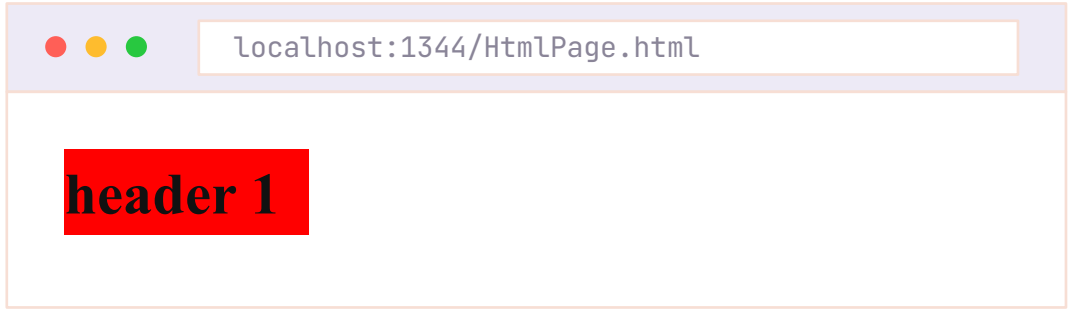
מה צבע הרקע של הכותרת?

```
.red {
    background-color: red;
}

h1 {
    background-color: yellow;
}

<h1 class="red">header 1</h1>
```

מה צבע הרקע של הכותרת?




```css
.red {
    background-color: red;
}

h1 {
    background-color: yellow;
}
```

```html
<h1 class="red">header 1</h1>
```

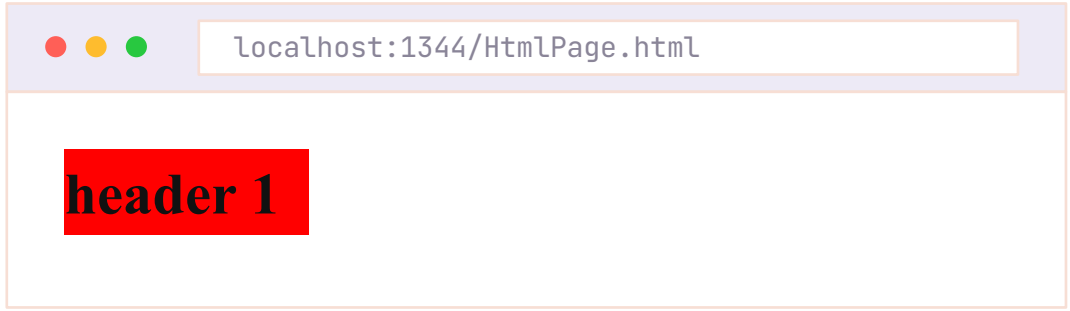
אדום. class חזק מתג -גם אם התג נרשם אחריו בקובץ.

ומה כשהעוצמה זהה?

```css
.red {
    background-color: red;
    color: blue;
}

.red {
    color: black;
}
```

ומה כשהעוצמה זהה?




```css
.red {
    background-color: red;
    color: blue;
}

.red {
    color: black;
}
```

- הרקע **אדום** - הוגדר פעם אחת בלבד.
- הצבע **שחור** - שתי ההגדרות באותה עוצמה ,ולכן האחרונה מנצחת.
- כשהעוצמה זהה - מי שמופיע אחרון בקובץ הוא זה שקובע.

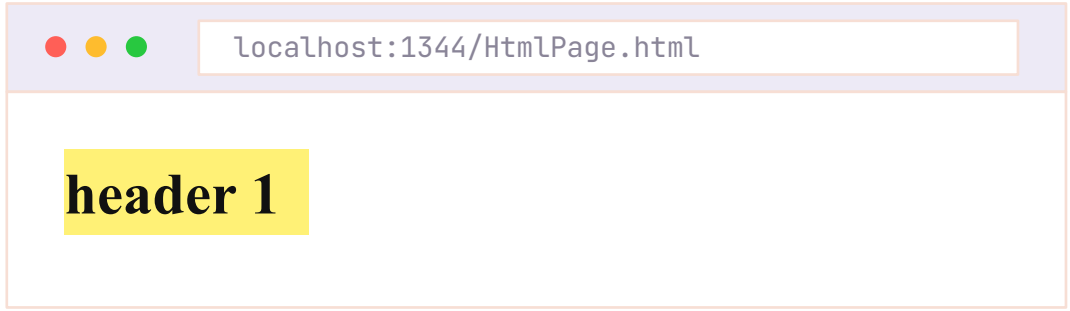
מה צבע הרקע עכשיו?

```
#myH1 {
    background-color: yellow;
}

.red {
    background-color: red;
}

<h1 class="red" id="myH1">header 1</h1>
```

מה צבע הרקע עכשיו?




```css
#myH1 {
    background-color: yellow;
}

.red {
    background-color: red;
}
```

```html
<h1 class="red" id="myH1">header 1</h1>
```

צהוב. id חזק מ-class.

ומי חזק מכולם?

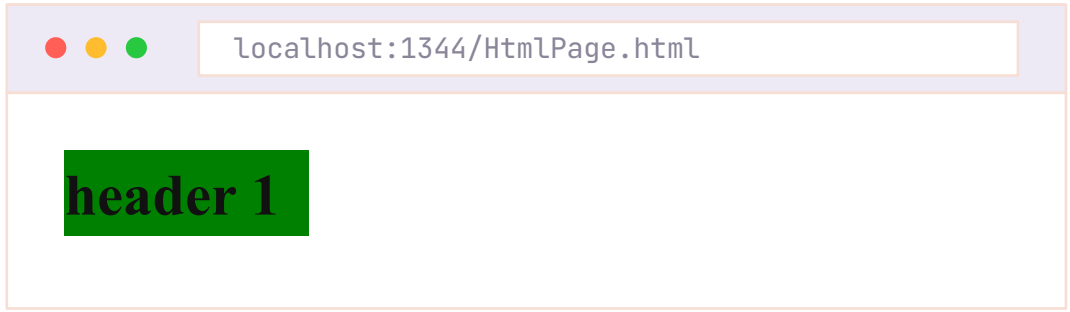
```html
<h1 class="red" id="myH1" style="background-color:green">header 1</h1>
```

```css
#myH1 {
    background-color: yellow;
}

.red {
    background-color: red;
}
```

ומי חזק מכולם?

```html
<h1 class="red" id="myH1" style="background-color:green">header 1</h1>
```




```css
#myH1 {
    background-color: yellow;
}

.red {
    background-color: red;
}
```

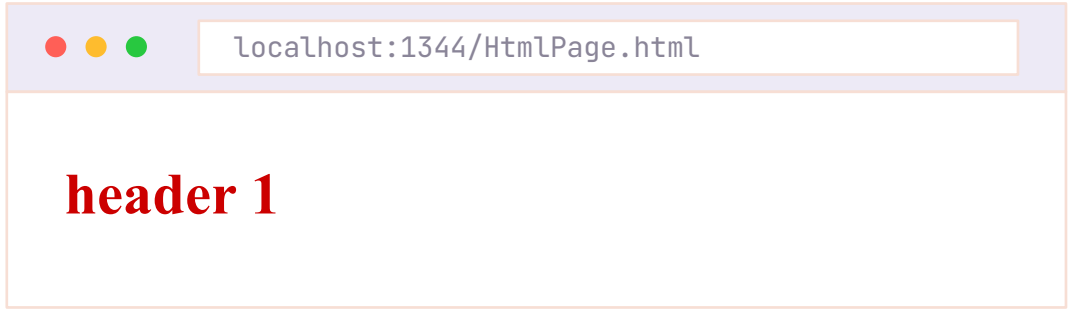
ירוק. style ברמת האלמנט חזק גם מ-id - ולכן כדאי להימנע ממנו.

מה קורה באלמנט עוטף?

```
#myDiv {
    color: red;
}


<div id="myDiv">
    <h1>header 1</h1>
</div>
```

הורשה מאלמנט עוטף




```css
#myDiv {
    color: red;
}
```

```html
<div id="myDiv">
    <h1>header 1</h1>
</div>
```

אדום. הצבע לא הוגדר על הכותרת -הוא עבר בירושה מה-`div` שהיא נמצאת בתוכו.

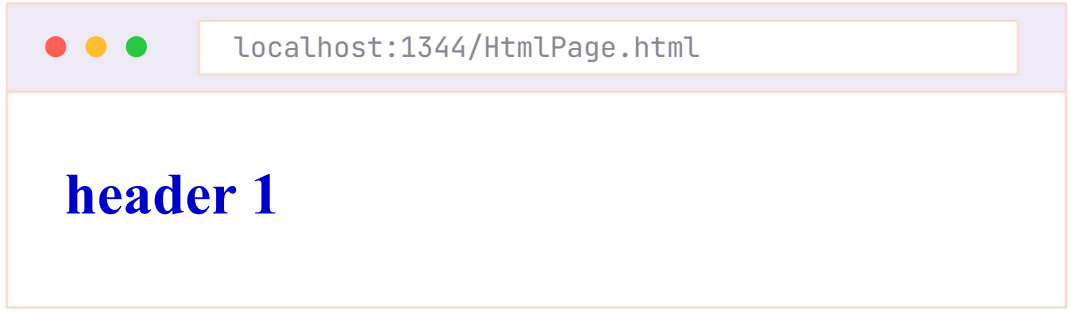
הורשה מול התייחסות ישירה

```
#myDiv {
    color: red;
}

h1 {
    color: blue;
}


<div id="myDiv">
    <h1>header 1</h1>
</div>
```

הורשה מול התייחסות ישירה




```css
#myDiv {
    color: red;
}

h1 {
    color: blue;
}
```

```html
<div id="myDiv">
    <h1>header 1</h1>
</div>
```

כחול. אמנם #myDiv חזק יותר מ-h1 - אבל הוא פונה ל-div, לא לכותרת. התייחסות ישירה תמיד מנצחת ירושה.

טבלת המשקלים

כל המקומות שבהם הגדרנו CSS מתחברים לגיליון אחד לפי סדר השורות .לאחר מכן מחושבת כל תכונה בנפרד:

1	10	100	1000
סלקטור תג h1	סלקטור class .red	סלקטור id #myH1	על האלמנט style style="..."

- התכונה עם המשקל הגבוה ביותר ,שממוקמת אחרונה -היא שתיושם.
- התייחסות ישירה לאלמנט תמיד חזקה יותר מהתייחסות עקיפה (ירושה).
- יש גם !important שדורס הכול -אבל השימוש בו נחשב לסימן לבעיה בעיצוב.

דוגמה מלאה

```
<style>
    #myDiv {
        color: red;
    }
    h1 {
        color: blue;
        background-color: yellow;
    }
    .headers {
        color: green;
    }
</style>

<div id="myDiv">
    <h1 class="headers">header 1</h1>
</div>
```

1 `#myDiv`

משקל - 100 אבל פונה ל-`div`, לא לכותרת. **ירושה**.

2 `h1`

משקל - 1 **התייחסות ישירה** לכותרת.

3 `.headers`

משקל - 10 גם היא **התייחסות ישירה**, וחזקה יותר.

דוגמה מלאה

```
<style>
    #myDiv {
        color: red;
    }
    h1 {
        color: blue;
        background-color: yellow;
    }
    .headers {
        color: green;
    }
</style>

<div id="myDiv">
    <h1 class="headers">header 1</h1>
</div>
```

1 `#myDiv`

משקל - 100אבל פונה ל-`div`, לא לכותרת. **ירושה**.

2 `h1`

משקל - 1**התייחסות ישירה** לכותרת.

3 `.headers`

משקל - 10גם היא **התייחסות ישירה**, וחזקה יותר.

התוצאה :ירוק .שתי ההתייחסויות הישירות גוברות על הירושה, ומביניהן `.headers` חזקה יותר.

04

סלקטורים מתקדמים

`:nth-child` `:hover` `>`

סקירה

`*`	כל האלמנטים בדף The Universal Selector
`div p`	כל `p` **בתוך** `div`, בכל עומק The Descendant Selector
`div > p`	רק `p` שהוא **ילד ישיר** של `div` The Child Selector
`p.intro`	רק `p` שיש לו `class="intro"` Element with Class
`input[type="text"]`	לפי ערך של תכונה The Attribute Selector
`a:hover`	רק כשהעכבר נמצא על האלמנט Pseudo-class

*

הסלקטור האוניברסלי

```
* {
    color: #000000;
}
```

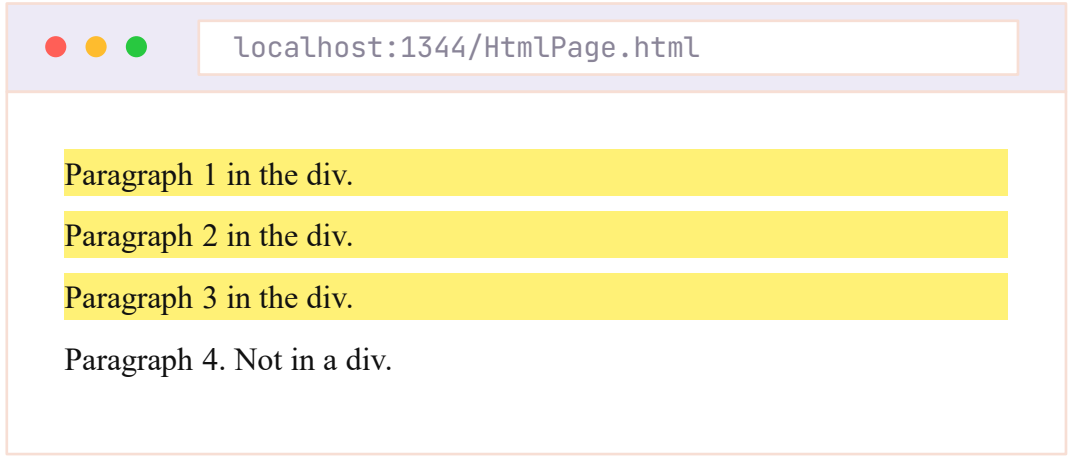
מה זה עושה

כל האלמנטים בדף יקבלו טקסט שחור.

שימוש נפוץ :איפוס `margin` ו-`padding` בתחילת הגיליון ,כדי להתחיל מנקודה אחידה בכל הדפדפנים.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

סלקטור צאצא

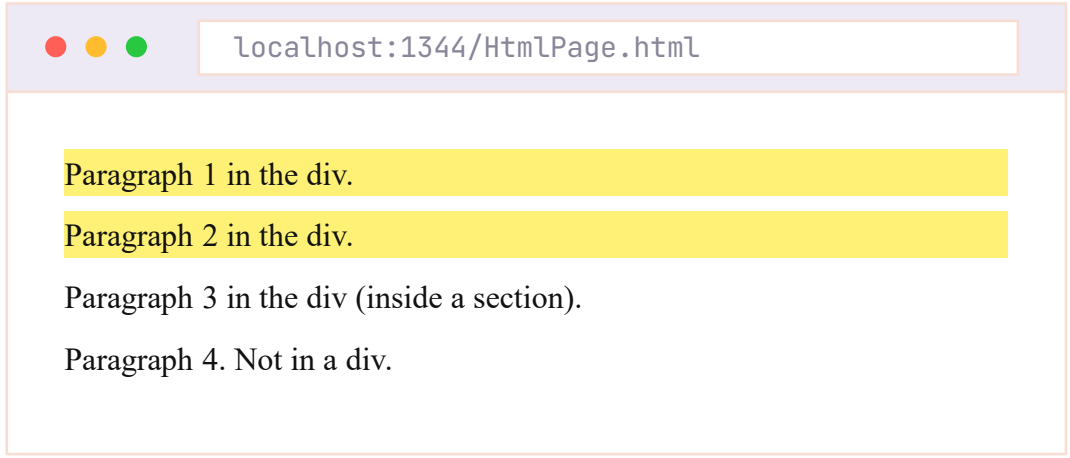



```css
div p {
    background-color: yellow;
}
```

```html
<div>
    <p>Paragraph 1 in the div.</p>
    <p>Paragraph 2 in the div.</p>
    <section>
        <p>Paragraph 3 in the div.</p>
    </section>
</div>
<p>Paragraph 4. Not in a div.</p>
```

הרווח פירושו בתוך - בכל עומק .גם פסקה 3 שנמצאת בתוך section בתוך ה-div, נצבעת.

סלקטור ילד ישיר

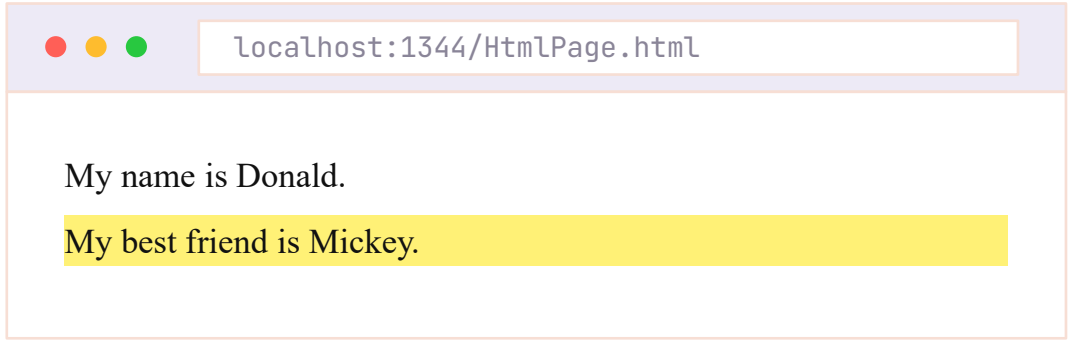



```css
div > p {
    background-color: yellow;
}
```

```html
<div>
    <p>Paragraph 1 in the div.</p>
    <p>Paragraph 2 in the div.</p>
    <section>
        <p>Paragraph 3 in the div.</p>
    </section>
</div>
<p>Paragraph 4. Not in a div.</p>
```

ההבדל היחיד :פסקה 3 לא נצבעת , כי היא נמצאת בתוך `section` - כלומר היא נכדה ,לא ילדה.

תגית עם מחלקה




```css
p.intro {
    background-color: yellow;
}
```

```html
<div class="intro">
    <p>My name is Donald.</p>
</div>

<p class="intro">My best friend is Mickey.</p>
```

נצבע רק ה-p שיש לו class="intro". ה-div עם אותה מחלקה לא נצבע - הוא לא p.

סלקטור לפי תכונה

```
input[type="text"] {
    color: red;
}
```

מה זה עושה

רק שדות `input` שהתכונה `type` שלהם היא `text` - ולא כפתורים או תיבות סימון.

שימושי מאוד בטפסים - נראה אותם בהרצאה הבאה.

:hover

משנה את העיצוב **רק** כשהעכבר נמצא מעל האלמנט:

```
a:hover {
    background-color: yellow;
}

p:hover, h1:hover {
    background-color: yellow;
}
```

`:hover` הוא `pseudo-class` - מחלקה מדומה שהדפדפן מפעיל לבד לפי מצב האלמנט.

יש עוד רבים כאלה: **`:focus`** כשהאלמנט נבחר, **`:visited`** קישור שביקרנו בו, **`:first-child`** ועוד.

:not()

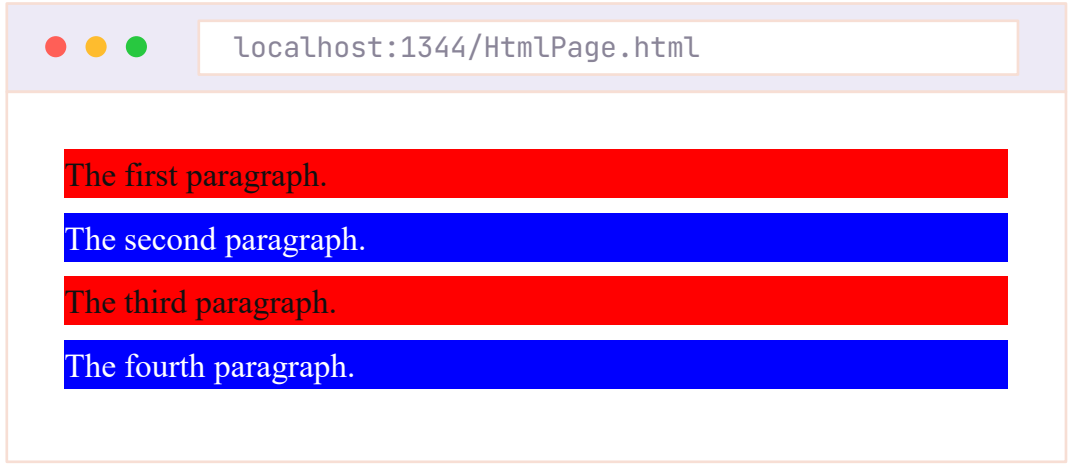



```
p {
    color: #000000;
}

:not(p) {
    color: #ff0000;
}
```

כל מה שאינו p נצבע באדום -הכותרת ,ה-div והקישור .הפסקאות נשארות שחורות.

:nth-child()




```css
p:nth-child(2) {
    background: red;
}

p:nth-child(odd) {
    background: red;
}

p:nth-child(even) {
    background: blue;
}
```

למעלה: `odd` ו-`even` - שורות מתחלפות בטבלה או ברשימה, בשתי שורות CSS.

רווח אחד שמשנה הכול

#header1.callout - בלי רווח

```
#header1.callout {
    color: red;
}


<div id="header1" class="callout">
    <p>this is p1</p>
    <p>this is p2</p>
</div>
```

אותו אלמנט: div שיש לו גם id="header1" וגם class="callout".

#header1 .callout - עם רווח

```
#header1 .callout {
    color: red;
}

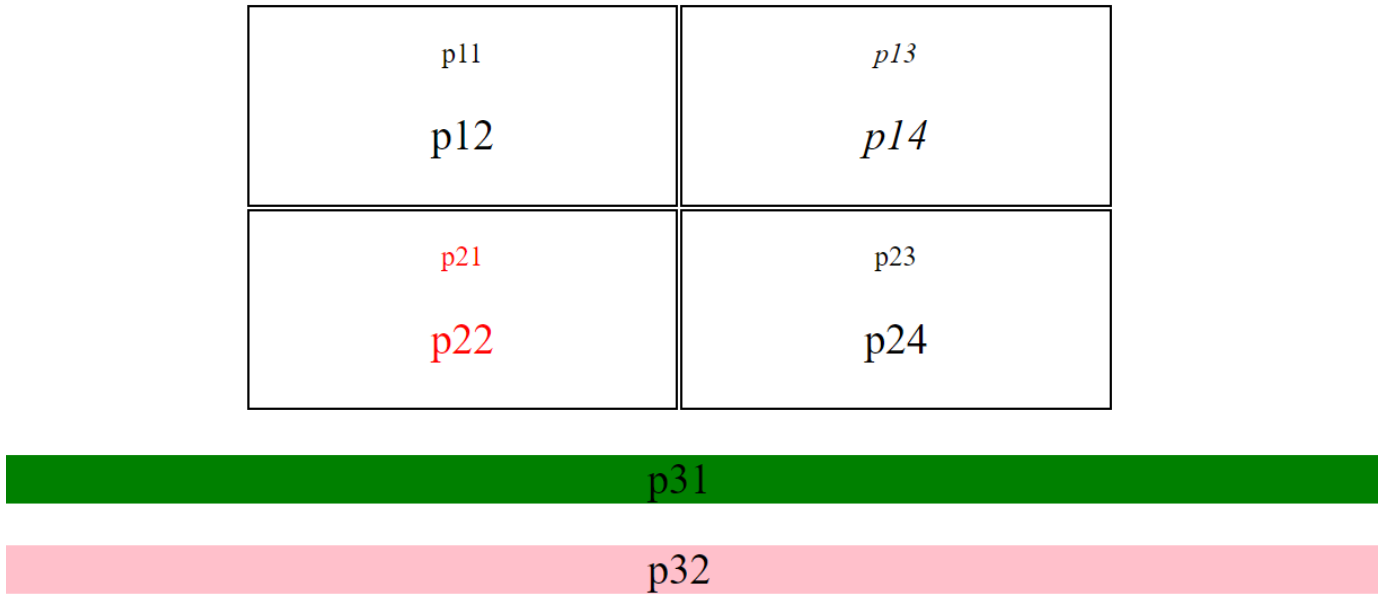

<div id="header1">
    <p class="callout">this is p1</p>
    <p>this is p2</p>
</div>
```

אלמנט **בתוך** אלמנט: p עם class="callout" שנמצא בתוך #header1.

רווח =בתוך. בלי רווח =אותו אלמנט . זו אחת הטעויות הנפוצות ביותר ב-CSS.

תרגיל

עבוד על קובץ ex.html css target שנה אותו לפי ההנחיות במסמך ה word שיראה בצורה הבאה



05

צבעים, גדלים וקבצים

צבע • px מול em • קובץ חיצוני

צבעים ב-CSS

```
p { color: green; }

p { color: rgb(0, 255, 0); }

p { color: #00fe00; }
```

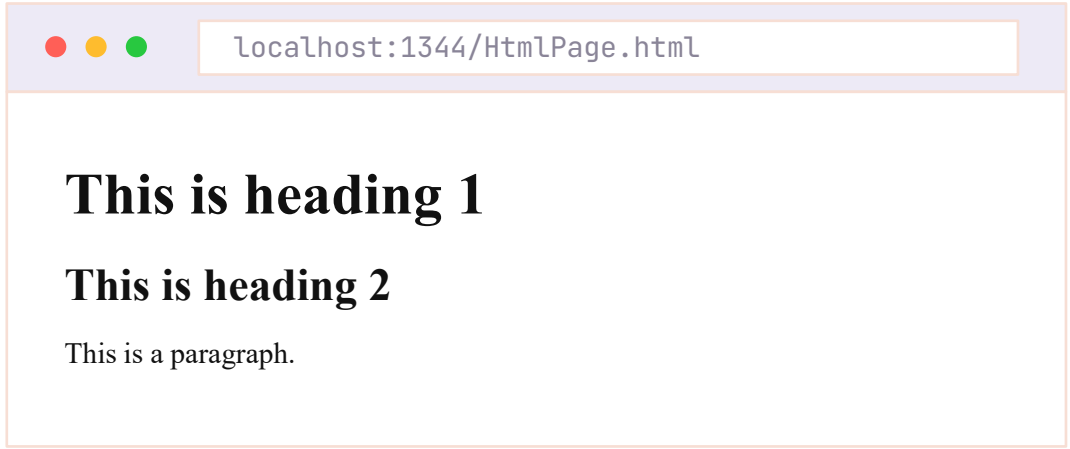
Color Names
כ 140-שמות צבעים מוכנים

RGB
מ-`rgb(0,0,0)` עד `rgb(255,255,255)`

Hexadecimal
מ-`#000000` עד `#FFFFFF`

RGB ו-Hex נותנים 16,777,216 אפשרויות -שמות הצבעים נותנים 140בלבד.

גודל טקסט ב-px




```css
h1 { font-size: 40px; }

h2 { font-size: 30px; }

p  { font-size: 14px; }
```

px הוא גודל קבוע :הוא לא משתנה גם אם המשתמש הגדיל את גודל הטקסט בדפדפן שלו.

גודל טקסט ב-em

`em` הוא **יחס** לגודל הטקסט של האלמנט העוטף. אם ברירת המחדל של הדפדפן היא `16px`:

```
h1 { font-size: 2.5em; }    /* 40px / 16 */
h2 { font-size: 1.875em; }  /* 30px / 16 */
p  { font-size: 0.875em; }  /* 14px / 16 */
```

היתרון

המשתמש יכול להגדיל את הטקסט בדפדפן, וכל הדף גדל איתו באופן יחסי - חשוב לנגישות.

שימו לב

`em` מצטבר: `em` בתוך `em` מוכפל. לכן היום נפוץ `rem`, שתמיד יחסי לשורש הדף.

הערות ומשתנים

הערות

```
/* This is a CSS comment.
   It can be broken onto multiple lines. */
```

ב-CSS יש רק סוג אחד של הערה - /* ... */. אין //.

משתנים

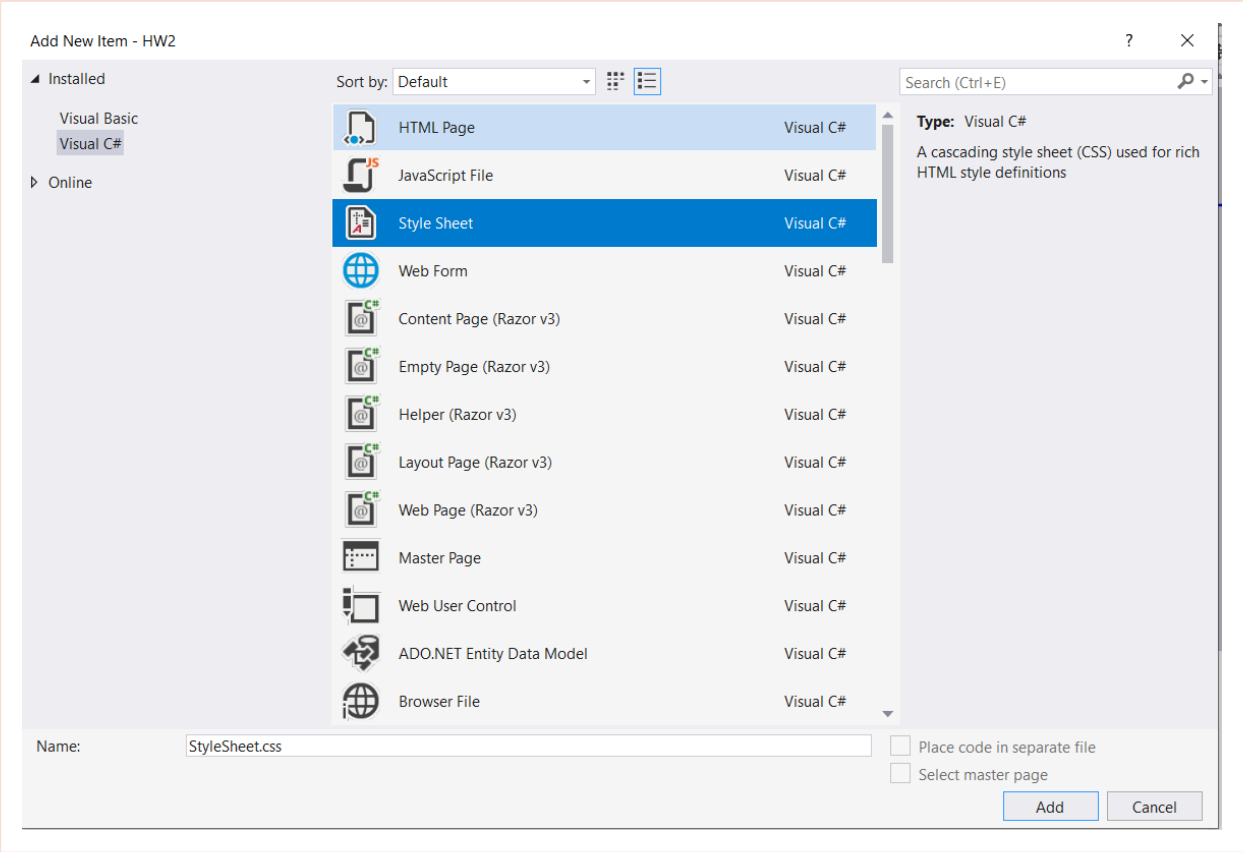
```
:root {
    --main-color: #E8532B;
}

button {
    background: var(--main-color);
}

a {
    color: var(--main-color);
}
```

משתנה מוגדר פעם אחת ב-:root, ומשמש בכל הקובץ. שינוי צבע המותג = שינוי שורה אחת.

הוצאת ה-CSS לקובץ חיצוני



- קליק ימני על הפרויקט ,ואז Add New Item>.
- בוחרים Style Sheet ונותנים שם - למשל StyleSheet.css.

- מעבירים לקובץ את כל מה שהיה בתוך תגית ה- <style>.

Solution Explorer

- Solution 'css example' (1 project)
  - css example
    - HtmlPage.html
    - **StyleSheet.css**

קישור הקובץ לדף

HtmlPage.html

```html
<head>
    <meta charset="utf-8" />
    <link href="StyleSheet.css" rel="stylesheet" />
</head>
```

StyleSheet.css

```css
#yael {
    color: red;
}

#yael1 {
    color: blue;
}

.h1BG {
    background-color: yellow;
}
```

- התגית `<link>` יושבת ב-`<head>`, והיא `void element` - אין לה תג סוגר.
- `href` מצביע על הקובץ, בדיוק כמו ב-`img` וב-`a`.
- אותו קובץ אחד יכול לשרת עשרות דפים -וזה כל היתרון.

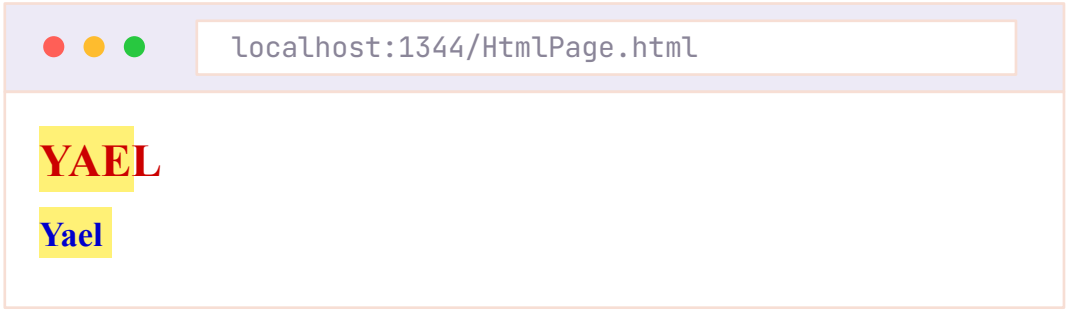
חיצוני מול פנימי

```
#yael {
    color: red;
}

#yael1 {
    color: blue;
}

.h1BG {
    background-color: yellow;
}
```

```
<head>
    <style>
        #yael {
            color: green;
        }
    </style>
    <link href="StyleSheet.css" rel="stylesheet" />
</head>
<body>
    <h1 id="yael" class="h1BG">YAEL</h1>
    <h2 id="yael1" class="h1BG">Yael</h2>
</body>
```



ה-<style> נכתב לפני ה-<link>, ולכן הקובץ החיצוני מגיע אחריו ומנצח -אלא אם נהפוך את הסדר.

לראות ולשנות CSS בדפדפן

F12
פותח את כלי הפיתוח

Inspect element
קליק ימני על אלמנט בדף – רואים את ה-HTML ואת ה-CSS שחלים עליו

להוסיף תכונה
אפשר לכתוב CSS חדש ולראות מיד את התוצאה

להוסיף מחלקה
אפשר לסמן ולבטל מחלקות בזמן אמת

השינויים ב-DevTools הם זמניים – הם נעלמים ברענון הדף. זה מגרש ניסויים, לא עורך.

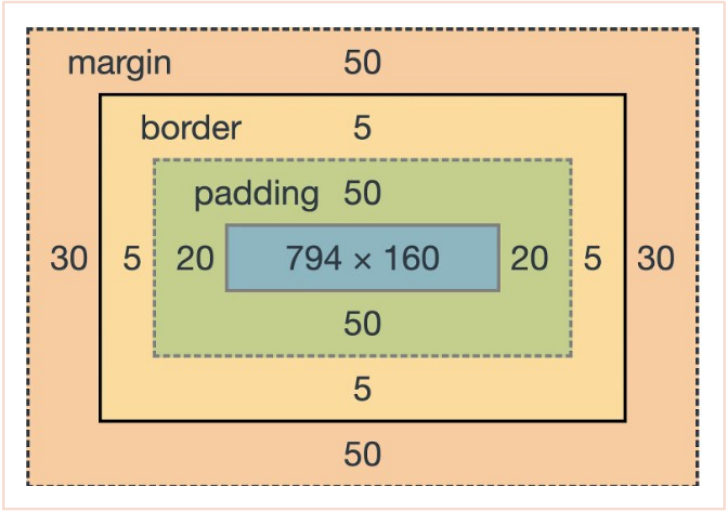
06

box-sizing וסמנטיקה

פריסה • אלמנטים סמנטיים • ARIA • SEO

הבעיה

- כפי שראינו בהרצאה 1ה-`Box Model` מוסיף את ה-`padding` וה-`border` **מעל** הרוחב שהגדרנו.
- נניח שרוצים 4תיבות, כל אחת `width: 25%` מרוחב הדף.
- אם נוסיף להן `padding` או `border` - הן כבר לא ייכנסו לשורה אחת.



שני המצבים

content-box - ברירת המחדל

הרוחב והגובה כוללים **רק את התוכן**.

ה-padding וה-border מתווספים מעליהם, והאלמנט תופס בפועל יותר מקום ממה שהגדרנו.

```
div {
    width: 100%;
    border: solid #5B6DCD 10px;
    padding: 5px;
}
```

border-box

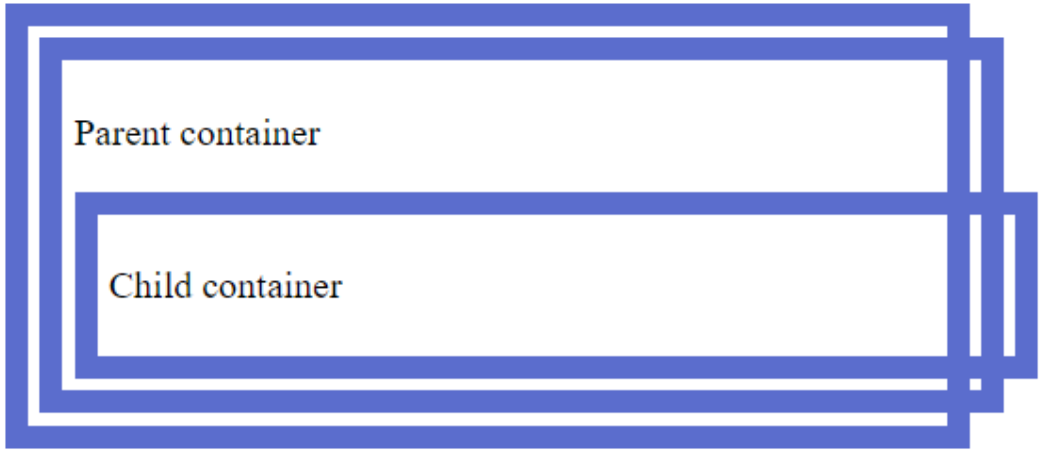
הרוחב והגובה כוללים את התוכן, את ה-padding **ואת** ה-border.

האלמנט נשאר בדיוק בגודל שהגדרנו -מה שמקל מאוד על תכנון פריסה.

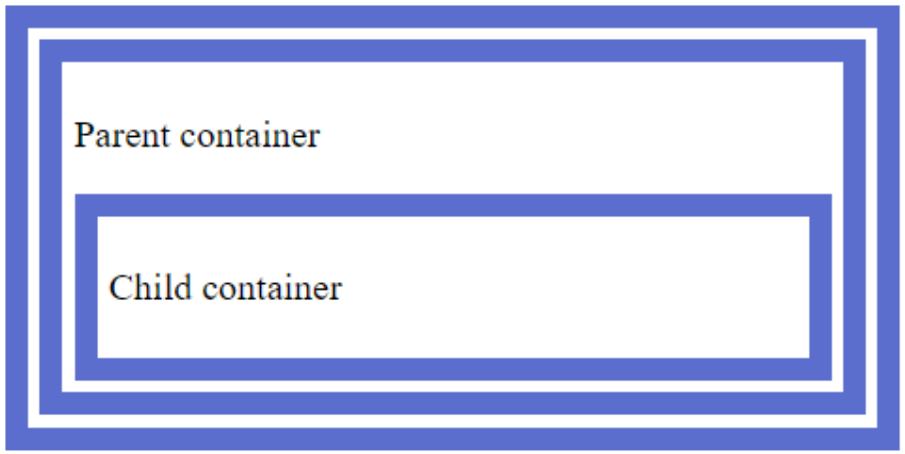
```
div {
    box-sizing: border-box;
    width: 100%;
    border: solid #5B6DCD 10px;
    padding: 5px;
}
```

אותו קוד, שתי תוצאות

`box-sizing: content-box`



`box-sizing: border-box`

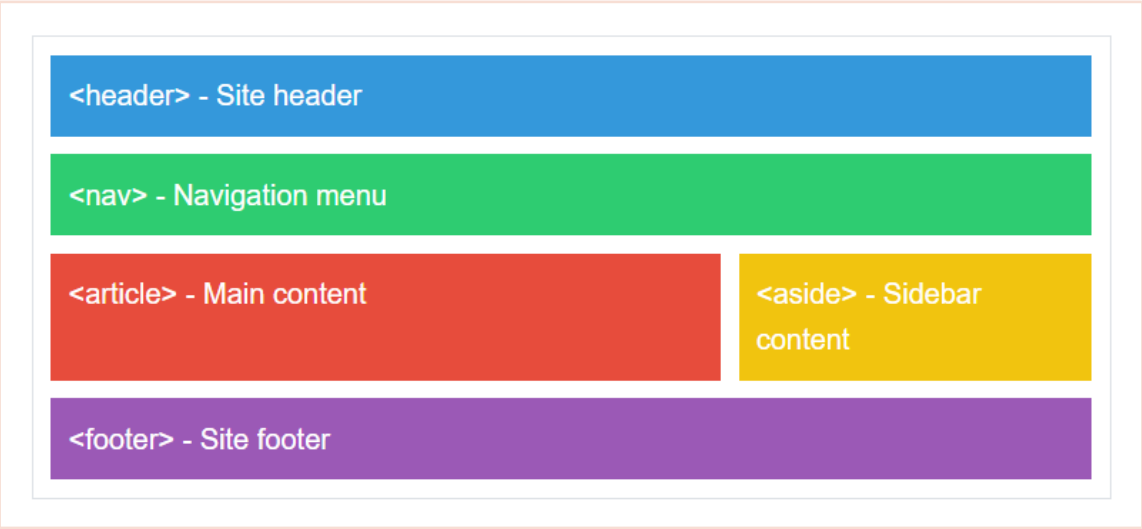


משמאל התיבה הפנימית "בורחת", כי ה-`border` נוסף מעל ה-`100%`. מימין היא נשארת בפנים.

לכן נפוץ מאוד להתחיל כל גיליון עיצוב עם `* { box-sizing: border-box; }`.

אלמנטים סמנטיים

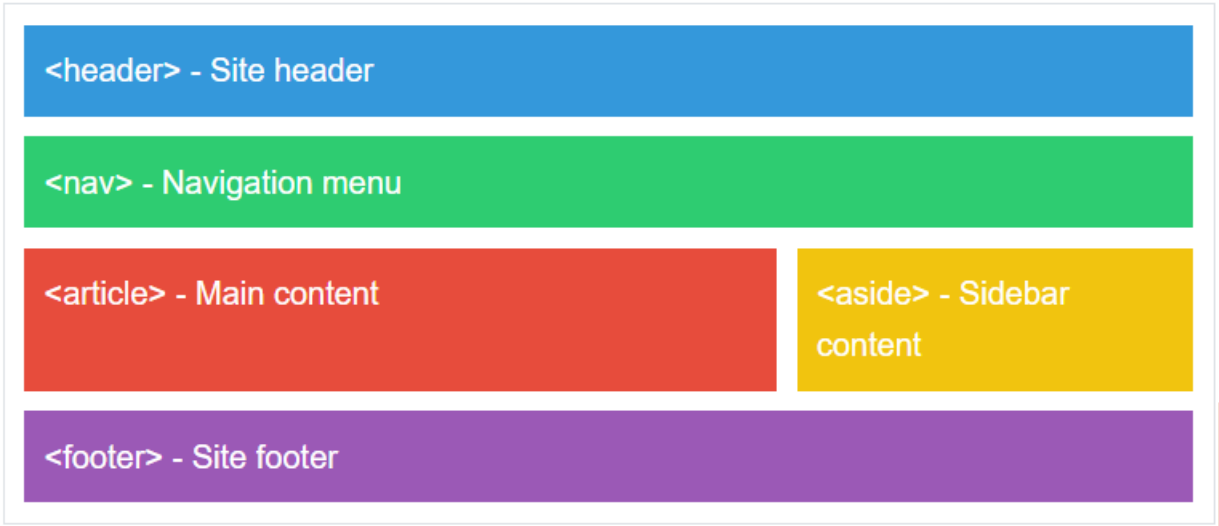
- אלמנטים שהשם שלהם **מתאר את התפקיד** שלהם בדף.
- מבחינת התצוגה הם מתנהגים בדיוק כמו `div`.
- הם נותנים מבנה ברור וקריא יותר -למתכנת ,לקוראי מסך ולמנועי חיפוש.



התגיות הסמנטיות

- `<header>` - כותרת האתר או של מקטע
- `<main>` - התוכן המרכזי של הדף -פעם אחת בלבד
- `<article>` - תוכן שעומד בפני עצמו -כתבה, פוסט
- `<footer>` - תחתית האתר או של מקטע
- `<nav>` - תפריט ניווט
- `<aside>` - תוכן צדדי -סרגל צד

- `<section>` - מקטע תוכן עם נושא משלו
- `<figure>` - תמונה עם כיתוב `<figcaption>`



אותו מבנה, שתי דרכים

עם div בלבד

```
<div id="header"></div>
<div class="section">
    <div class="article">
        <div class="figure">
            <img>
            <div class="figcaption"></div>
        </div>
    </div>
</div>
<div id="footer"></div>
```

עם אלמנטים סמנטיים

```
<header></header>
<section>
    <article>
        <figure>
            <img>
            <figcaption></figcaption>
        </figure>
    </article>
</section>
<footer></footer>
```

התצוגה זהה לחלוטין - אבל השני מסביר את עצמו, ומנועי חיפוש וקוראי מסך מבינים אותו.

ARIA - נגישות

- הנגשת אתרים לאנשים עם מוגבלויות: קוראי מסך , עיוורון צבעים ,קושי מוטורי ,ניווט במקלדת בלבד.
- המטרה :שהאתר יהיה **שמיש לכולם**.
- **ARIA** הוא סט של תכונות שמוסיפים ל-**HTML** כדי לעזור לקוראי מסך להבין את המבנה והמשמעות.

```
<button aria-label="סגור חלון">X</button>
```

בלי **aria-label** ,קורא מסך יקריא רק "X" - "והמשתמש לא יידע מה הכפתור עושה .עם התכונה ,הוא יקריא "סגור חלון."

SEO - קידום במנועי חיפוש

איך גורמים לגוגל להבין את האתר? בעיקר באמצעות מבנה HTML נכון:

סמנטי HTML

`<footer>`, `<main>`, `<nav>`, `<header>`

כותרות היררכיות

`h1` אחד בדף, ומתחתיו `h2`, `h3` בסדר הגיוני

`meta description`

תיאור קצר שמופיע בתוצאות החיפוש

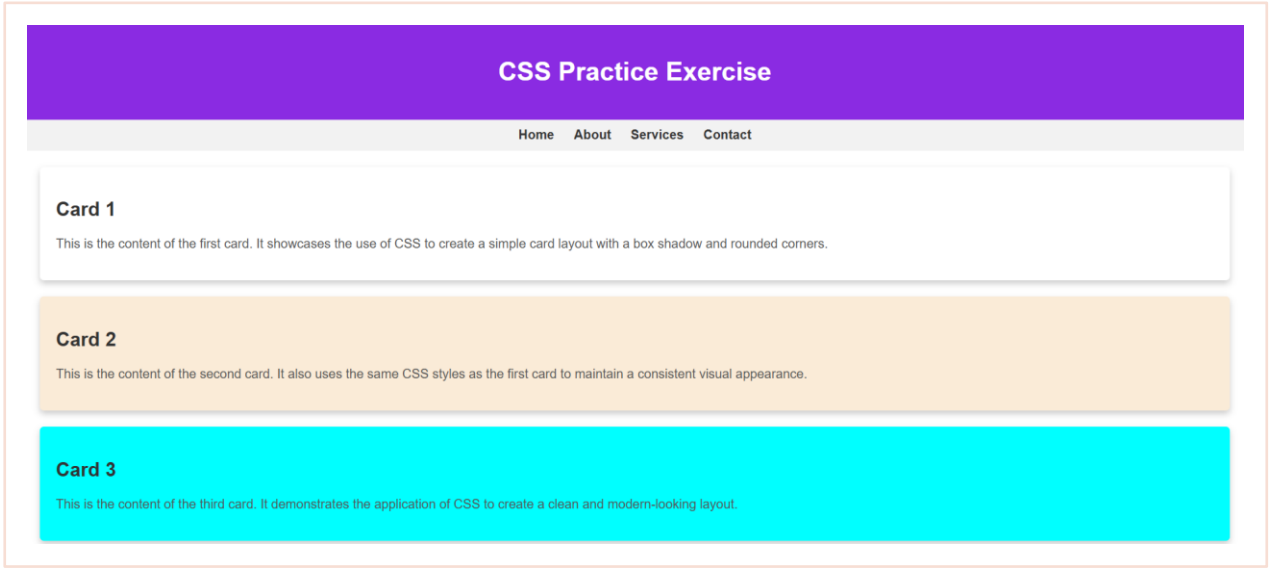
`alt` לתמונות

טקסט חלופי - חשוב גם ל-SEO וגם לנגישות

```
<meta name="description" content="client side course" />
```

תרגיל כיתה מתקדם

- נתון לכם קובץ שלד -**אין** לשנות או להוסיף אלמנטים.
- מחלקות ה-CSS כבר קיימות בקובץ ;עליכם למלא רק את התכונות של כל מחלקה.
- התוצאה הסופית מוצגת משמאל.
- רמז :תצטרכו `:hover` ו-`:nth-child`.



להעמיק

CSS Zen Garden

אותו קובץ HTML בדיוק, עם עשרות גיליונות עיצוב שונים – ההדגמה הטובה ביותר לכוח של CSS.

csszengarden.com/pages/alldesigns

MDN Web Docs

התיעוד הרשמי והמלא ביותר ל-CSS, בעברית ובאנגלית.

developer.mozilla.org/docs/Web/CSS

תודה!

בהרצאה הבאה :טפסים ו-Bootstrap

צד לקוח • יעל סלע זעירא